

Relatório Técnico

Sistema de Certificados Acadêmicos

Análise de arquitetura, backend, frontend, banco de dados e segurança

Data: 15 de junho de 2026 | **Escopo:** diagnóstico (nenhum código alterado) | **Stack:** FastAPI · React/Vite · SQLite · Pillow

Resumo executivo

A fundação técnica é boa: separação limpa de responsabilidades, geração de PDF sofisticada, geração de código segura, SQLite em modo WAL para concorrência e validações de upload de imagem corretas. O sistema, porém, **ainda não está pronto para uso real** por três motivos centrais: **(1) ausência total de autenticação** no admin; **(2) o modelo da planilha não captura evento/carga horária por aluno** (grava o curso no campo "evento"); e **(3) reprocessar a planilha gera certificados duplicados**. Somam-se a falta de uma área administrativa de consulta do histórico e a exposição pública de dados pessoais.

1. Visão geral do sistema

Monorepo com **dois aplicativos Python/FastAPI independentes** que compartilham um **mesmo banco SQLite** e a **mesma pasta de PDFs**. A integração ocorre por import direto do pacote database (via bootstrap de `sys.path`) e por caminhos resolvidos uma única vez em `database/db.py`.

Projeto	Stack	Porta	Papel
certificados-admin/backEnd	FastAPI + Pillow + pandas	8000	Gera PDFs, grava no banco, valida código
certificados-admin/frontend	React + Vite + TS + Tailwind + Zustand + Fabric.js	5173	UI da secretaria (gerar, validar, editor, templates)
certificados-consulta	FastAPI + Jinja2	8001	Site público (busca por nome/código, download)
database/	SQLite (WAL)	—	Camada de acesso compartilhada (db.py)
storage/pdfs/	Filesystem	—	PDFs gerados

A arquitetura conceitual está clara e bem documentada (o README é exemplar). O problema é que a **implementação cobre apenas uma fração dos requisitos funcionais** e tem lacunas críticas de segurança.

2. Funcionalidades já implementadas

Backend (admin) — main.py

- GET /health, /courses, /templates.
- POST /upload-template — PNG/JPG por curso, com validação de extensão, **magic bytes** e limite de 5 MB.
- POST /generate-certificates — recebe .xlsx + texto + data + template_id opcional; gera PDFs em lote.

- POST /download-certificates — ZIP com proteção contra *path traversal*.
- GET /validate/{code} — validação por código.
- Router de **templates visuais** — CRUD completo + upload de background validado.

Geração de certificado — generator.py

- Renderização sobre PNG-base com Pillow, salvando PDF a 300 dpi.
- Fontes proporcionais à altura, *fit-to-width*, *fit-to-space* e **justificação completa** — bem feito.
- **Código de validação impresso no certificado.**
- Modo "template visual" alternativo dirigido por layout JSON.
- Geração de código **colisão-safe** com secrets e alfabeto sem caracteres ambíguos — **destaque positivo**.

Frontend (admin)

- 4 abas: Gerar, Validar, Editor visual, Gerenciar templates.
- Formulário + upload com validação de .xlsx e estados de loading/erro/sucesso bem tratados.
- Lista com **botão copiar código**, visualizar e baixar individual/ZIP.
- Editor visual com Fabric.js (canvas, propriedades, mock data, grid, zoom).

Consulta pública e Banco

- Busca por **nome (parcial)** e por **código**; download do PDF por código com nome sanitizado.
- Tabela certificates com unique_code UNIQUE, índice em nome e created_at.
- Modo **WAL** para leitura/escrita concorrente — **destaque positivo**.
- Testes: test_normalize.py cobre normalização de data e texto (único módulo).

3. Funcionalidades incompletas ou ausentes

#	Funcionalidade requerida	Estado
1	Autenticação da área administrativa	Ausente — nenhuma rota protegida
2	Validar dados da planilha antes de gerar	Sem etapa de pré-validação/preview
3	Visualizar certificados já emitidos (admin)	Mostra só os da sessão atual (Zustand)
4	Consultar por nome/curso/evento (admin)	Não existe no admin
5	Dados por aluno (evento, carga horária, datas)	Planilha só lê nome, email, curso; resto é texto único igual p/ todos
6	Evitar duplicatas / idempotência	Reprocessar a planilha gera novos códigos e linhas
7	QR code no certificado	Apenas "preparado" e nunca gerado
8	Carga horária como campo estruturado	Embutida no texto livre
9	Paginação / rate limiting na busca pública	Ausente
10	Revogação / reemissão / exclusão	Ausente
11	Auditoria (quem emitiu, quando)	Sem usuários e sem log de ação
12	Validação do curso contra lista canônica	read_participants aceita qualquer string
13	Limite de tamanho/linhas da planilha	Lê o arquivo inteiro em memória sem limite
14	Testes de reader/generator/db/endpoints	Só há testes de normalização

#	Funcionalidade requerida	Estado
15	Deploy de produção (HTTPS, proxy, hardening)	Bind em 0.0.0.0 com CORS *

4. Problemas encontrados

CRÍTICOS

P1 — Nenhuma autenticação no admin. Não há Depends, token, sessão ou senha. A API escuta em 0.0.0.0:8000 com CORS allow_origins=["*"]. Qualquer pessoa na rede pode gerar certificados, subir templates e baixar todos os PDFs. Viola o requisito de "controle mínimo de autenticação".

P2 — Modelo de dados da planilha não atende ao requisito. O leitor exige apenas nome, email, curso (reader.py). Evento, carga horária e datas vêm de um único campo texto_certificado aplicado identicamente a todos. Pior: event_name gravado no banco é o curso, não o evento — então "consultar por evento" é impossível.

P3 — Duplicatas silenciosas / sem idempotência. Cada geração cria códigos aleatórios novos; o INSERT OR IGNORE só evita colisão de unique_code (que nunca colide). Reenviar a mesma planilha cria linhas e PDFs duplicados. O legado certificates.json é a prova: os mesmos 4 alunos aparecem ~5 vezes.

P4 — Admin não lista nem busca o histórico. A aba "Gerar" mostra só o resultado da sessão (Zustand). Não há endpoint nem tela de consulta por nome/curso/evento/código no admin.

IMPORTANTES

P5 — Exposição de dados pessoais / LGPD no site público. A busca por nome retorna todos os certificados que casam (nome, curso, data, código) e o download exige só o código, que é exibido no resultado. Qualquer pessoa pode enumerar nomes e baixar PDFs de terceiros, sem captcha nem rate limit.

P6 — curso não validado na planilha. Diferente do upload de template (que valida contra COURSES), a geração aceita qualquer curso. String fora da lista cai no template padrão e grava lixo em event_name.

P7 — livro/folha/linha sem persistência nem sentido. Calculados por lote, sempre reiniciam (livro=1), não são salvos no banco e não aparecem no PDF. Lógica de negócio pela metade.

P8 — Sem limite de tamanho/linhas do .xlsx. await file.read() carrega tudo em memória e gera 1 PDF por linha sem teto — risco de exaustão de memória/DoS.

P9 — Concorrência no store de templates visuais. visual_template_store.py faz read-modify-write de um JSON sem lock nem escrita atômica (corrida pode corromper). Ironicamente, template_store.py já faz isso certo.

P10 — Dados institucionais e datas hardcoded. ISSUE_LOCATION, SIGNATORY_NAME e SIGNATORY_TITLE fixos no código. E MONTH_NAMES está sem acentos ("marco", não "março") — sai impresso "24 de marco de 2026".

MENORES / DÍVIDA TÉCNICA

- **P11** — Artefatos legados/mortos: certificates.json e a pasta output/certificados/ (PDFs agora vão para storage/pdfs).
- **P12** — Bootstrap de sys.path repetido em 5+ módulos, substituindo empacotamento adequado. Frágil.
- **P13** — Duplicação frontend/backend: COURSES e normalizeCourse em TS e Python; /courses existe mas não é usado.
- **P14** — Catches silenciosos no frontend (.catch(() => {})) escondem falhas do usuário.
- **P15** — /validate não distingue "inválido" de "erro": retorna {valid:false} em qualquer falha.

- **P16** — QR "preparado" e descartado: `_prepare_for_future_qr_code` monta um dict que ninguém usa.

5. Melhorias recomendadas

Segurança

- Autenticação no admin (JWT ou sessão+cookie HttpOnly) com Depends em toda rota não-pública. *Sem isso o sistema não vai a produção.*
- Restringir CORS a origens conhecidas; não fazer bind em 0.0.0.0 sem proxy/HTTPS.
- Rate limit + paginação na busca pública; exigir código para download (não permitir baixar via busca por nome).
- Limite de tamanho e de linhas do .xlsx; ler com openpyxl em modo read-only/streaming.

Banco e rastreabilidade

- Normalizar o modelo: aluno, evento/curso, certificado, usuario_admin, com FKs.
- Chave de idempotência (UNIQUE(aluno, evento, data) ou hash) para impedir duplicatas no reprocessamento.
- Auditoria: issued_by, status (ativo/revogado) e tabela de log.

Geração

- Carga horária, evento e data por linha da planilha + preview/validação antes de gerar.
- QR code apontando para consulta/?codigo=CERT-... no PDF. *Validação moderna e à prova de digitação.*
- Dados institucionais/assinante via configuração (env/DB/template); corrigir acentos dos meses.

Arquitetura / qualidade

- Empacotar com pyproject.toml e instalação editável para eliminar o hack de sys.path.
- Migrar os JSON-stores de template para o banco (ou ao menos dar lock+atomic ao visual store).
- Front consumir /courses em vez de duplicar a lista; centralizar tipos.
- Cobertura de testes (reader, generator, db, endpoints, colisão de código) + CI.
- Remover artefatos legados (certificates.json, output/certificados/).

6. Modelo ideal de funcionamento

SECRETARIA (autenticada)

1. Login -> token/sessao
2. Upload da planilha (.xlsx): nome, cpf?, curso, evento, carga_horaria, data_inicio, data_fim
3. PRE-VALIDACAO: backend le, valida tipos/obrigatorios/curso, mostra PREVIEW (linhas OK / com erro) -- nada e gravado ainda
4. Secretaria confirma -> geracao em lote:
 - aloca codigo unico CERT-ANO-XXXXXX
 - checa idempotencia (nao reemite duplicata sem flag)
 - renderiza PDF (template do curso ou visual) com QR + codigo
 - grava no banco (issued_by, status=ativo) + PDF em storage/
5. Retorna lista com codigo + link; secretaria CONSULTA o historico por nome/curso/evento/codigo, reemite ou revoga.

ALUNO (publico)

6. Busca por codigo (recomendado) ou nome
7. Ve dados validados (nome, curso/evento, carga, data, status)
8. Baixa o PDF / escaneia o QR -> pagina de validacao

7. Sugestão de estrutura de banco de dados

```
-- Usuarios administrativos
CREATE TABLE admin_users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,          -- bcrypt/argon2
  role TEXT NOT NULL DEFAULT 'secretaria',
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Eventos / cursos (catalogo)
CREATE TABLE events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,                  -- "IX Campeonato Code Race"
  course TEXT NOT NULL,                -- curso (lista canonica)
  workload_hours INTEGER,              -- carga horaria
  start_date TEXT, end_date TEXT,
  UNIQUE(name, course, start_date)
);

-- Alunos (melhora rastreabilidade)
CREATE TABLE students (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL, email TEXT,
  doc_id TEXT,                        -- CPF/matricula (cuidado LGPD)
  UNIQUE(doc_id)
);

-- Certificados
CREATE TABLE certificates (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  unique_code TEXT NOT NULL UNIQUE,    -- CERT-2026-AB1234
  student_id INTEGER REFERENCES students(id),
  event_id INTEGER REFERENCES events(id),
  participant_name TEXT NOT NULL,       -- snapshot (imutavel)
  event_name TEXT NOT NULL,             -- snapshot
  workload_hours INTEGER,               -- snapshot
  issue_date TEXT NOT NULL,
  pdf_path TEXT NOT NULL,
  certificate_text TEXT,
  status TEXT NOT NULL DEFAULT 'ativo', -- ativo|revogado
  issued_by INTEGER REFERENCES admin_users(id),
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE(participant_name, event_name, issue_date) -- idempotencia
);
CREATE INDEX idx_cert_name  ON certificates(participant_name);
CREATE INDEX idx_cert_event ON certificates(event_name);
CREATE INDEX idx_cert_status ON certificates(status);

-- Auditoria
CREATE TABLE audit_log (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  actor_id INTEGER REFERENCES admin_users(id),
  action TEXT NOT NULL,                 -- generate|revoke|reissue|login
  details TEXT,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

Mantém snapshots (nome/evento/carga) no certificado para imutabilidade legal, mesmo que o catálogo mude depois. Os PDFs continuam em storage/, referenciados por pdf_path.

8. Sugestão de endpoints da API

Método	Rota	Finalidade	Acesso
POST	/auth/login	Login da secretaria	Público
POST	/auth/logout	Encerrar sessão	Admin

Método	Rota	Finalidade	Acesso
GET	/courses	Lista de cursos	Admin
POST	/certificates/validate-spreadsheet	Pré-validar planilha (preview, sem gravar)	Admin
POST	/certificates/generate	Gerar lote (após confirmação)	Admin
GET	/certificates	Listar/buscar emitidos (paginado)	Admin
GET	/certificates/{code}	Detalhe de um certificado	Admin
POST	/certificates/{code}/revoke	Revogar	Admin
POST	/certificates/download-zip	Baixar vários em ZIP	Admin
*	/visual-templates/{id}	CRUD de templates visuais	Admin
POST	/templates/upload	Template PNG por curso	Admin
GET	/public/verify/{code}	Validar por código (dados + status)	Público
GET	/public/search?nome=	Busca por nome (paginada, rate-limited)	Público
GET	/public/certificates/{code}/download	Download do PDF	Público
GET	/health	Healthcheck	Público

9. Prioridades de implementação

CRÍTICO — bloqueia uso real

- Autenticação + proteção das rotas admin; restringir CORS/bind (P1).
- Idempotência / detecção de duplicatas no reprocessamento (P3).
- Tela + endpoint admin de listagem e busca de emitidos (P4).
- Definir e implementar o modelo de dados por aluno (evento/carga/data por linha) (P2).
- Mitigar exposição pública / LGPD: rate limit, paginação, download só por código (P5).

IMPORTANTE

- Validar curso contra a lista canônica na planilha (P6).
- Limite de tamanho/linhas da planilha + leitura segura (P8).
- QR code no PDF apontando para a validação pública (P16).
- Persistir auditoria (issued_by, status, log) (P7).
- Corrigir acentos dos meses e externalizar dados institucionais (P10).
- Lock/atOMICidade ou migração do store de templates visuais (P9).

MELHORIAS FUTURAS

- Empacotamento Python (eliminar sys.path hack) (P12).
- Front consumir /courses; remover duplicação de tipos (P13).
- Remover artefatos legados (certificates.json, output/) (P11).
- Cobertura de testes ampla + CI; erros mais granulares no front (P14/P15).
- Configuração de deploy (HTTPS, proxy reverso, ASGI gerenciado).

10. Checklist final

Segurança

- ☐ Login + hash de senha (argon2/bcrypt) para a secretaria
- ☐ Depends(auth) em todas as rotas admin
- ☐ CORS restrito + bind sem 0.0.0.0 em produção / atrás de proxy HTTPS
- ☐ Rate limiting e paginação na busca pública
- ☐ Download público apenas por código; revisar exposição por nome (LGPD)
- ☐ Limite de tamanho e de linhas do .xlsx
- ☐ Variáveis sensíveis só via .env

Funcionalidades

- ☐ Pré-validação/preview da planilha antes de gerar
- ☐ Colunas por aluno: curso, evento, carga horária, datas
- ☐ Validação de curso contra COURSES
- ☐ Idempotência (chave de negócio única) contra duplicatas
- ☐ Tela admin: listar/buscar emitidos por nome/curso/evento/código
- ☐ Revogação/reemissão de certificado
- ☐ QR code no PDF -> página pública de validação
- ☐ Página pública dedicada de validar por código com status

Banco

- ☐ Migrar para modelo normalizado (alunos, eventos, certificados, usuários, auditoria)
- ☐ Persistir issued_by e status
- ☐ Índices em event_name e status
- ☐ Mover stores de template JSON para o banco

Qualidade

- ☐ Corrigir acentuação dos meses (março, etc.)
- ☐ Externalizar local/assinante do certificado
- ☐ Empacotar projetos (pyproject) e remover bootstrap de sys.path
- ☐ Front usa /courses; remover COURSES/normalizeCourse duplicados
- ☐ Remover certificates.json e output/certificados/
- ☐ Testes para reader, generator, db, endpoints e geração de código
- ☐ CI (lint + testes)

Documentação

- ☐ Especificar formato exato da planilha (colunas, exemplos)
- ☐ Documentar fluxo de autenticação e deploy de produção
- ☐ Documentar política de retenção/LGPD dos PDFs